



Testing VPC Connectivity



MHM Aslam

```
ec2-user@ip-10-0-0-226 ~]$ curl https://learn.nextwork.org/projects/aws-host-a-website-on-s3
!DOCTYPE html>
html lang="en">
<head>
  <meta charset="UTF-8" />

  <title>NextWork - Host a Website on Amazon S3</title>
  <meta content="Let's host your very own website on Amazon S3!" name="description" />
  <meta
    content="NextWork - Host a Website on Amazon S3"
    property="og:title"
  />
  <meta
    content="Let's host your very own website on Amazon S3!"
    property="og:description"
  />
  <meta
    content="NextWork - Host a Website on Amazon S3"
    property="twitter:title"
  />
  <meta
    content="Let's host your very own website on Amazon S3!"
    property="twitter:description"
  />

  <meta content="/static/og-project.png" property="og:image" />
```



Introducing Today's Project!

What is Amazon VPC?

It is useful because it gives you complete control over your cloud networking environment, allowing you to securely launch resources (like EC2 instances) in an isolated virtual network that you design — similar to a traditional on-premises data center — while leveraging AWS's scalable infrastructure. This enables you to control inbound/outbound traffic, separate public-facing resources from private ones, and enforce security boundaries.

How I used Amazon VPC in this project

In today's project, I used Amazon VPC to create a complete isolated network environment with both public and private subnets, an internet gateway for outbound internet access, custom route tables to direct traffic, security groups and network ACLs to control access, and EC2 instances in each subnet. I then tested connectivity — from my local machine to the public instance (using EC2 Instance Connect), from the public instance to the private instance (using ping), and from the public instance to the internet (using curl) — to verify that all VPC components were configured correctly.

One thing I didn't expect in this project was...

One thing I didn't expect in this project was that even though my public instance had a route to the internet gateway and a public IP, I still couldn't connect via EC2 Instance Connect until I manually added the SSH inbound rule to the security group — I assumed the service would automatically handle that. It taught me that AWS gives you full control, which means you're responsible for every open port.



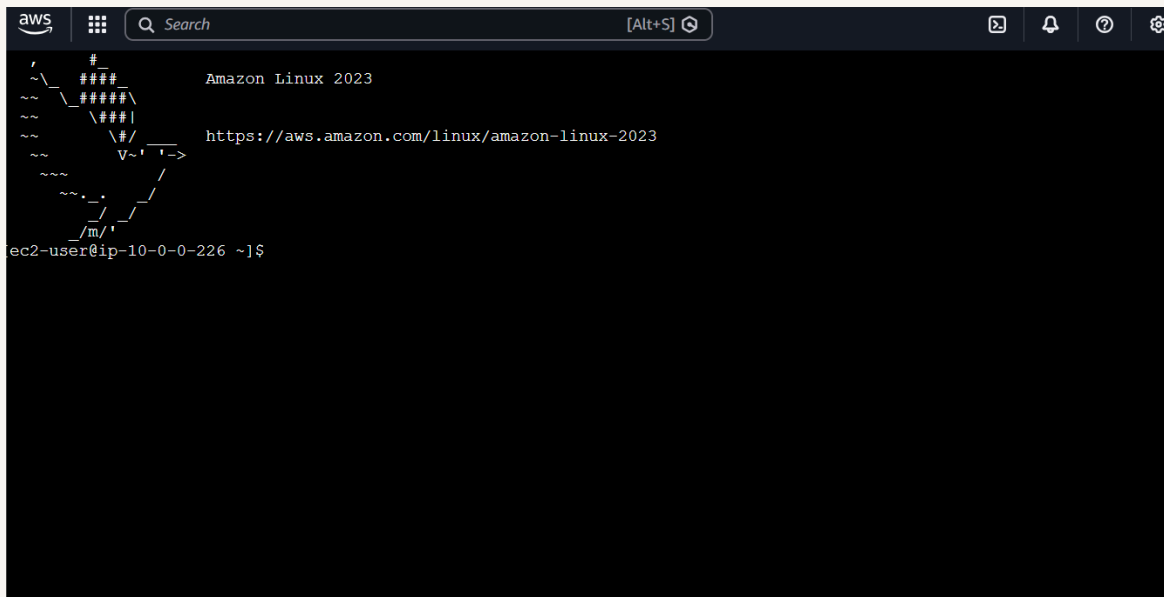
This project took me...

This project took me about 45 minutes to an hour, including reading through the instructions, creating the private subnet and route table, launching the EC2 instances, and troubleshooting the two connection issues (missing SSH inbound rule for Instance Connect, and missing ICMP rules for ping between instances). The hands-on debugging probably added an extra 10–15 minutes, but that was the most valuable part — it really solidified how security groups, NACLs, and route tables work together.

Connecting to an EC2 Instance

Connectivity means the ability to successfully send and receive data between two points (e.g., you and your public EC2 instance). In this project, good connectivity means SSH works, your public instance reaches the internet, and your private instance doesn't (unless via NAT). Bad connectivity means something in routing, security groups, or ACLs is blocking the path.

My first connectivity test was whether I could connect to the NextWork Public Server (the EC2 instance in my public subnet) using EC2 Instance Connect from the AWS Management Console.



i-09b87883f6bd9f90e (NextWork Public Server)



EC2 Instance Connect

EC2 Instance Connect is a service that lets you securely SSH into your EC2 instances directly from the AWS Management Console (or CLI) without manually generating and managing key pairs. It creates a one-time key pair for you, pushes the public key to the instance, and removes it after a short period.

My first attempt at getting direct access to my public server resulted in an error, because the security group attached to my EC2 instance was missing an inbound rule allowing SSH (port 22) from my IP address (or from the EC2 Instance Connect service range). Without that rule, the security group blocked the connection before it could reach the instance.

I fixed this error by adding an inbound rule to the security group allowing SSH (port 22) from the appropriate source (my IP address or the EC2 Instance Connect CIDR range). Once the rule was added, the connection succeeded.



A circular profile picture of a person with dark hair, wearing a blue shirt, standing in front of a colorful background.

Connectivity Between Servers

I used ping to test the connectivity between my public EC2 instance and my private EC2 instance. If ping succeeds, it confirms that network routes, security groups, and ACLs allow traffic between the two subnets. If it fails (which may happen here without proper ICMP rules), it signals a blockage I need to troubleshoot.

The ping command I ran was `ping <private-IPv4-address-of-the-private-EC2-instance>` (e.g., `ping 10.0.1.45`). This sends ICMP echo requests from the public server to the private server's internal IP address to test basic network reachability between the two subnets.

The first ping returned no response (timeout or "Request timed out"). This meant that the ICMP echo requests from the public server were being blocked — likely by the private subnet's Network ACL or the private EC2 instance's security group, both of which needed an inbound rule allowing ICMP (IPv4) traffic from the public server's IP or subnet.



MHM Aslam

NextWork Student

nextwork.org

```
~\      #_
~\      ###_
~\      #####\
~\      \###l
~\      \#/
~\      v~'  _->
~\      .-.-.-
~\      /m/'  _-
ec2-user@ip-10-0-0-226 ~]$ ping 10.0.1.42
PING 10.0.1.42 (10.0.1.42) 56(84) bytes of data.
```

Troubleshooting Connectivity

I troubleshooted this by adding an inbound ICMP (IPv4) rule to the private instance's security group (allowing Echo Request from the public subnet's CIDR) and updating the private subnet's Network ACL to allow ICMP traffic (both directions, with proper ephemeral port handling). After applying these changes, the ping succeeded.

```
ec2-user@ip-10-0-0-226 ~]$ ping 10.0.1.42  
PING 10.0.1.42 (10.0.1.42) 56(84) bytes of data.  
cC  
--- 10.0.1.42 ping statistics ---  
152 packets transmitted, 0 received, 100% packet loss, time 157072ms
```

```
ec2-user@ip-10-0-0-226 ~]$ ping 10.0.1.42  
PING 10.0.1.42 (10.0.1.42) 56(84) bytes of data.  
64 bytes from 10.0.1.42: icmp_seq=1 ttl=127 time=3.14 ms  
64 bytes from 10.0.1.42: icmp_seq=2 ttl=127 time=0.460 ms  
64 bytes from 10.0.1.42: icmp_seq=3 ttl=127 time=0.482 ms  
64 bytes from 10.0.1.42: icmp_seq=4 ttl=127 time=0.530 ms  
64 bytes from 10.0.1.42: icmp_seq=5 ttl=127 time=0.456 ms  
64 bytes from 10.0.1.42: icmp_seq=6 ttl=127 time=0.440 ms  
64 bytes from 10.0.1.42: icmp_seq=7 ttl=127 time=0.446 ms  
64 bytes from 10.0.1.42: icmp_seq=8 ttl=127 time=0.494 ms  
64 bytes from 10.0.1.42: icmp_seq=9 ttl=127 time=0.532 ms  
64 bytes from 10.0.1.42: icmp_seq=10 ttl=127 time=0.540 ms  
cC  
--- 10.0.1.42 ping statistics ---  
10 packets transmitted, 10 received, 0% packet loss, time 9306ms  
rtt min/avg/max/mdev = 0.440/0.752/3.144/0.797 ms  
ec2-user@ip-10-0-0-226 ~]
```


A circular profile picture of a person with dark hair, wearing a blue shirt, standing in front of a green background with some text and graphics.

Connectivity to the Internet

Curl is a command-line tool that transfers data to or from a server using protocols like HTTP, HTTPS, or FTP. In this project, I used curl to test internet connectivity from my public EC2 instance — for example, `curl https://google.com` — to see if the instance can successfully reach external websites, which confirms that the route table and internet gateway are working correctly.

I used curl to test the connectivity between my public EC2 instance and the internet (e.g., external web servers like google.com). Unlike ping (which tests basic ICMP reachability), curl uses HTTP/HTTPS to verify that actual web traffic can go out through the internet gateway and receive a response — confirming that outbound internet access works for real applications.

Ping vs Curl

Ping and curl are different because ping uses ICMP (Internet Control Message Protocol) to test basic network reachability (layer 3), while curl uses HTTP/HTTPS (over TCP) to test actual data transfer from web servers (layer 7). Ping checks if a host is alive; curl checks if a web service responds correctly.



Connectivity to the Internet

I ran the curl command `curl https://learn.nextwork.org/projects/aws-host-a-website-on-s3` which returned an HTTP 200 OK response along with the HTML content of that webpage — confirming that my public EC2 instance could successfully reach external web servers over the internet via the internet gateway.

```
ec2-user@ip-10-0-0-226 ~]$ curl https://learn.nextwork.org/projects/aws-host-a-website-on-s3
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />

  <title>NextWork - Host a Website on Amazon S3</title>
  <meta content="Let's host your very own website on Amazon S3!" name="description" />
  <meta
    content="NextWork - Host a Website on Amazon S3"
    property="og:title"
  />
  <meta
    content="Let's host your very own website on Amazon S3!"
    property="og:description"
  />
  <meta
    content="NextWork - Host a Website on Amazon S3"
    property="twitter:title"
  />
  <meta
    content="Let's host your very own website on Amazon S3!"
    property="twitter:description"
  />

  <meta content="/static/og-project.png" property="og:image" />
```



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

